

LocalHouseLLM Research

Independent AI Systems & Local Inference Research Division

The Hidden Layer

Report 02 · Niche & Under-the-Radar Techniques | Prepared by LocalHouseLLM Research | Confidential
Technical Compendium

The Hidden Layer: Niche Techniques for Local LLM Inference

LocalHouseLLM Research

Executive Summary

Beyond the well-known LLM optimizations (FlashAttention, GPTQ/AWQ, llama.cpp, etc.) lies a hidden layer of niche techniques that squeeze every ounce of efficiency from local inference. In this deep dive we found a raft of under-the-radar methods — from radical KV-cache eviction and compression tricks to novel speculative decoding schemes, from dynamic token-skipping hacks to exotic quantization and kernel-fusion tactics. Many of these ideas appear only in workshop papers, obscure repos or blog comments, and they often hinge on clever approximations or hardware tricks. Examples include HashEvict (an LSH-based, pre-attention eviction strategy), Counter-Causal Surprise (a cutting-edge KV eviction using “future-only” attention), LazyLLM (dynamic per-token KV pruning during prefill), and SpecDiff (a speculative decoding variant using discrete diffusion to parallelize token generation). We also discovered Token Recycling (reusing candidate tokens in a BFS-like draft tree) and DecoQuant (outlier-aware low-bit KV quantization via tensor decomposition) that most practitioners have never heard of.

These methods tend to solve very specific bottlenecks (e.g. KV memory bloat or decoding latency) and often rely on unusual algorithmic tricks. They matter because they push LLM inference farther than standard tools: some yield 2x or more speedups or 90%+ KV compression in certain settings. However, they are extremely obscure — usually unfinely-coded, only a single lab has tried them, or they require custom kernels. We rate evidence for these techniques as weak or speculative unless independently confirmed.

In short, the “deep underground” optimization layer is a dense web of micro-hacks: LSH eviction for KV caches, graph-level kernel fusions, speculative draft-model hybrids, lazy token-skipping at prefill, aggressive decompositions, and more. The goal is an intelligence briefing that leaves no stone unturned in local AI inference.

Taxonomy of Obscure Optimization Families

- **KV-Cache Eviction & Compression** — Techniques that discard or condense stored key/value tensors to cap memory. Examples: heavy-hitter eviction (H2O),

LSH-based eviction (HashEvict), counter-causal surprise eviction, low-rank projections (Palu and its successor FDC), channel-shrinking (CSKV), and product-quantization (PQCache).

- **Speculative Decoding Variants** — Unusual “draft-and-verify” schemes beyond standard speculative decoding, including draft models using RNNs or diffusion instead of transformers, parallel draft generation, etc. E.g. CTC-Drafter, Clover-2 (RNN-based drafter), Sorted Spec-Decoding (S2D), SpecDiff (discrete diffusion drafts), and Token Recycling (BFS reuse tree).
- **Dynamic Token Pruning / Early-Exit** — Methods that skip tokens or layers on the fly. LazyLLM prunes KV computation of irrelevant prompt tokens during prefill.
- **Exotic Quantization & Calibration** — Beyond GPTQ/AWQ, including outlier suppression and transform tricks. DecoQuant (matrix-decomposition for KV quantization), OffQ (Hadamard-rotation outlier offset for uniform low-bit quantization), SpinOut (injecting artificial outliers for better rotation learning), and PQCache.
- **Memory Paging / Offload Hacks** — Unusual schemes to spill KV or activations to CPU/SSD; mostly speculative blog/forum musings (SSD-backed KV, multi-device tiling).
- **Custom Kernel / Graph Fusion** — Compiler/runtime tricks such as the Mirage-Persistent Kernel (MPK), which fuses an entire multi-GPU inference into one “megakernel,” reducing overhead.

Method Entries

HashEvict — KV Eviction/Compression (Attention-free via LSH). A 2024 arXiv method where keys/queries are projected to short binary LSH codes, and at each decode step the token with highest Hamming distance to the current query is evicted. Requires only cheap bitwise operations (no full attention) and achieves ~30–70% KV compression with minor quality loss. Trick: SimHash precomputation to rank tokens by cosine similarity cheaply. Why obscure: Very new (Dec 2024), only arXiv workshop, minimal code. Hardware: GPU-friendly (bitwise ops). Maturity: Research prototype. Weakness: Eviction is greedy and myopic. Evidence: Moderate; a single paper with promising results.

Counter-Causal Surprise — KV Eviction (Prediction-based). A novel 2026 idea by Gould & van den Hengel that scores tokens by how predictable they are from the future context, not the past. Runs a single-layer “counter-causal” attention (each token attends only to future tokens) and measures log-probability (surprise); low-surprise tokens are evicted. Why obscure: Only on arXiv (July 2026), non-trivial implementation. Weakness: Extra forward pass overhead. Evidence: Weak; new preprint with one evaluation, no third-party reproduction yet.

CSKV (Channel-Shrinking KV) — KV Compression (Architecture-level). Proposed 2024, exploits redundancy in the embedding dimension of KV via low-rank decomposition of each layer’s key/value weights, caching only the low-rank versions. Uses a “bi-branch” cache: small recent-window full-precision KV plus larger low-dimension KV for the rest. Claims ~80% KV memory reduction with minimal accuracy loss. Why obscure: Only in a NeurIPS workshop paper (Sept 2024). Evidence: Moderate; solid experiments, code available.

LazyLLM — Dynamic Token Pruning (Prefill-stage). A 2024 arXiv method that dynamically decides which past prompt tokens matter for the next token, skipping KV computation for unimportant tokens during prefill and decoding. Uses an auxiliary

“importance network.” Reported 2.3x speedup in prefill on a 7B model. Weakness: some accuracy drop; overhead of deciding tokens. Evidence: Moderate; one paper with ablation.

CTC-Drafter — Speculative Decoding (CTC-based draft model). An early 2024 arXiv introduces a spec-dec strategy using CTC (Connectionist Temporal Classification) to train a draft model that outputs tokens in parallel rather than autoregressively. Why obscure: First to apply CTC in this context; no code except the author’s GitHub. Weakness: CTC models can be tricky to optimize; acceptance rate may be low. Evidence: Weak; only one ArXiv and single-authored GitHub.

Clover-2 — Speculative Decoding (RNN-based draft). Follow-up to “Clover”; uses a lightweight RNN as the draft model, distilling knowledge to narrow the accuracy gap between RNN and transformer drafts. Why obscure: Not formally published, just arXiv/tech report. Evidence: Weak (self-reported, no independent benchmarks).

Sorted Speculative Decoding (S2D) — Speculative Decoding (multi-model draft). Addresses serving multiple target models: sorts/matches smaller drafts to multiple LLMs simultaneously, pre-generating candidate tokens sorted by estimated likelihood, then verifying across models. Why obscure: Niche scenario (multi-model servers), early arXiv. Evidence: Weak; single ArXiv with concept and results on a multi-model benchmark.

Speculative Diffusion (SpecDiff) — Speculative Decoding (Diffusion-based draft). A 2024 NAACL-published idea: use a discrete diffusion model to generate an entire draft sequence in parallel, non-causally, then the LLM verifies them. Benefit: Massive parallelism (up to 7.2x speedup claimed). Weakness: Requires training/running a diffusion model (heavy). Evidence: Moderate (peer-reviewed, with experiments), but adoption unknown.

Token Recycling — Speculative Decoding (Draft-tree BFS). A 2024/25 ACL paper notes many candidate tokens reappear; stores tokens suggested by any previous decoding step in a graph, and every new token triggers a BFS-like expansion of candidate sequences. Trick: tiny adjacency matrix to track co-occurrences and reuse them. Benefit: ~2x speedup on all tested models with <2 MB overhead. Evidence: Moderate (strong result, code in SpecBench tool).

DecoQuant — KV-Cache Quantization (Tensor Decomposition). A 2024 arXiv method for extreme low-bit KV compression. Decomposes each KV matrix into “small” and “large” parts; the large (low-rank) part is quantized in 2–4 bits, the small part retains high precision. Benefit: ~75% KV memory reduction reported. Weakness: Adds decomposition step at prefill time. Evidence: Moderate.

PQCache — KV-Cache Compression (Product Quantization). A 2024 arXiv that treats keys as an embedding retrieval problem: applies product quantization to keys, clustering per head/layer and storing only centroids and codes. During inference, PQ indexes estimate which past tokens are relevant. Benefit: maintains quality while greatly reducing direct KV storage; improved QA scores with low latency. Weakness: PQ dequantization adds overhead. Evidence: Moderate; single preprint, no known re-implementations.

Palu (Low-Rank KV Projection) — KV-Cache Compression (Low-rank). A 2024 ICLR poster & arXiv (July 2024) that factorizes each KV layer weight into low-rank matrices, decompressing on-the-fly during attention, with fused kernels to hide decompression latency. Benefit: ~50–90% KV reduction, with <1% accuracy loss.

Evidence: Good (open code, multiple models tested).

FDC (Fast KV Dimensionality Compression) — KV-Cache (Adaptive Dimensionality). A 2024 arXiv follow-up to Palu by the same group. Removes Palu's decompression overhead by modifying the attention kernel itself, and adapts compression ratio per head/layer. Benefit: Reported 1.97x throughput and 64% job completion reduction vs Palu. Weakness: Requires custom CUDA kernels. Evidence: Weak (only one preprint so far).

OffQ — Outlier Suppression (Quantization). A June 2026 arXiv (Tencent/Tsinghua) tackling large activation outliers. Observes outliers lie in a low-dimensional subspace across layers; identifies that subspace via top-1 PCA, rotates activations so outliers concentrate in few channels, then does a Hadamard rotation to absorb these outliers as zero-point offsets. Result: subsequent uniform 4-bit quantization can proceed as if outliers didn't exist. Benefit: Enables full 4-bit (W4A4KV4) quantization with uniform multipliers. Evidence: Moderate (detailed arXiv with ablation, no external tests yet).

Mirage Persistent Kernel (MPK) — Graph-Level Kernel Fusion (GPU). A 2025 CMU/UW project that compiles an entire inference process (across layers and GPUs) into a single "persistent kernel" that never exits, eliminating launch overhead and overlapping communication with computation. Yields up to ~6x latency speedups. Weakness: Very complex (requires a whole new runtime and toolchain). Evidence: Moderate (demonstrated in blog with prototype code).

Lineage Maps

KV Eviction/Compression: Early ideas (2023) include H2O (heavy-hitter tokens) and L2-dropout. Later, TOVA and Scissorhands explored static deletion, leading to adaptive policies. Mid-2024, Palu introduced low-rank KV compression. Late 2024, CSKV (channel low-rank) and HashEvict (LSH) emerged. In 2025–26, FDC refined Palu's approach and Counter-Causal Surprise proposed a new predictive score.

Speculative Decoding: Standard line was SpecDec (2022–23) and Speculative Sampling. In 2023 Medusa, Spectral, etc. tuned draft models. In 2024, CTC-Drafter appeared; later Clover-2 distilled RNN drafts, and mid-2024 saw Sorted SpecDec (S2D) and Token Recycling. Late-2024 NAACL introduced SpecDiff (diffusion).

Quantization & Transform: Outlier work started with Outlier Suppression+ in early 2023, QuaRot/SpinQuant rotations in 2023. OffQ builds on that lineage. DecoQuant (May 2024) and SpinOut extend rotation ideas into KV-specific transforms.

Kernel/Compiler: The fusion idea has roots in GPU persistent kernels and the HPC community. Mirage-MPK cites works on pipelining/triton.

Comparison Within Families

KV Eviction (HashEvict, Counter-Causal, H2O, TOVA): Counter-Causal is cleverest (uses model introspection) but slow (extra pass). H2O has strong theoretical motivation but is simple. HashEvict is straightforward LSH. Ease of deploy: HashEvict is easiest. Adoption: H2O/TOVA have code in vLLM forks; HashEvict could be added by advanced users; Counter-Causal is very experimental.

Speculative Decoding (CTC vs Clover2 vs TokenRec vs SpecDiff): SpecDiff (7x parallel) is most novel; TokenRec (BFS reuse) surprisingly effective; CTC and Clover-2

are incremental. Ease: CTC is relatively simple if you have a CTC model; TokenRec needs a draft tree but no extra training; SpecDiff requires a full diffusion model.

Dynamic Pruning: Only LazyLLM stands out. It outperforms fixed pruning by adapting each step, but adds controller overhead.

Quantization/Transforms (DecoQuant vs OffQ vs PQCache): Different problems (KV vs activation). OffQ is about preserving uniform quant kernels via offsets. DecoQuant is simpler (block quantization after decomposition). PQCache is compress/lookup rather than quant. All are experimental research code requiring custom kernels.

Kernel/Compiler (MPK): No direct competitor; the main comparison is normal execution or torch.compile. MPK beats them but at the cost of retooling.

Ratings

- **HashEvict:** Stability experimental; Obscurity niche; Usefulness high; Novelty medium; HW medium; Integration hard; Adoption low.
- **Counter-Causal Surprise:** Stability research prototype; Obscurity deep underground; Usefulness potential; Novelty high; HW low; Integration difficult; Adoption very low.
- **CSKV:** Stability experimental; Obscurity niche; Usefulness moderate; Novelty medium; HW medium; Integration hard; Adoption low.
- **LazyLLM:** Stability research prototype; Obscurity niche; Usefulness high (2x prefill speed); Novelty medium; HW low; Integration medium; Adoption low.
- **CTC-Drafter:** Stability experimental; Obscurity obscure; Usefulness medium; Novelty high; HW low; Integration hard; Adoption very low.
- **Clover-2:** Stability experimental; Obscurity niche; Usefulness low-to-medium; Novelty low; HW low; Integration medium; Adoption very low.
- **SpecDiff:** Stability experimental; Obscurity obscure; Usefulness high (7x speed); Novelty high; HW high; Integration very hard; Adoption very low.
- **Token Recycling:** Stability experimental; Obscurity niche; Usefulness high (2x speed, ~0 overhead); Novelty high; HW low; Integration hard; Adoption low.
- **DecoQuant:** Stability experimental; Obscurity niche; Usefulness high (75% KV reduction); Novelty medium; HW medium; Integration hard; Adoption low.
- **PQCache:** Stability experimental; Obscurity niche; Usefulness medium; Novelty medium; HW medium; Integration hard; Adoption low.
- **Palu:** Stability usable but niche; Obscurity niche; Usefulness high (50–90% KV compress); Novelty medium; HW medium; Integration hard; Adoption low.
- **FDC:** Stability experimental; Obscurity obscure; Usefulness high (doubles throughput); Novelty high; HW high; Integration very hard; Adoption very low.
- **OffQ:** Stability experimental; Obscurity niche; Usefulness high (makes 4-bit practical); Novelty high; HW medium; Integration hard; Adoption very low.
- **MPK (Mirage):** Stability experimental; Obscurity niche; Usefulness high (major speedups multi-GPU); Novelty high; HW high; Integration very hard; Adoption very low.

Abandoned / Fork-only Projects

- **SpecBench's TokenRetire** — an early draft-based caching hack, later evolved into Token Recycling.

- **Speculative Phrasal Decoding** — a forgotten blog idea about drafting entire phrases using n-grams; no formal follow-up known.
- **GPU LRU eviction prototype** — an experimental true-LRU policy on KV cache; code lived in a private repo and died.
- **Forks of Palu** — a few GitHub forks with improved “rank search,” none matured beyond initial repos.
- **MinicoreFocus** — a GitHub prototype LLM inference engine with a hard-coded “layer pruning schedule”; never integrated into any library.
- **Onnx/kernels** — some outdated PRs on ONNX Runtime adding quantization/paging support (2019–2021) that never merged.

Hidden Gems Worth Testing

- **HashEvict** — easy concept, strong results; worth prototyping in a fork of vLLM or transformer-attention code.
- **LazyLLM** — surprisingly simple idea to skip KV; could be built via iterative attention masks at prefill.
- **Token Recycling** — if you have a multi-pass inference setup, its BFS reuse might double speed with little risk.
- **PQCache** — product quantization of KV is underexplored; worth testing on language retrieval tasks.
- **DecoQuant** — the mixed-bit scheme for KV is clever; implementing it could slash GPU memory.
- **OffQ** — possibly the single best trick to make true 4-bit inference feasible on new hardware.

Most Promising but Overlooked

- **Counter-Causal Surprise Eviction:** Technically neat and model-agnostic; could be combined with others.
- **FDC (Fast Dim Compression):** If integrated, promises ~2x throughput.
- **SpecDiff:** If someone packs a good diffusion model, 7x gains are huge — revolutionary for high-end local setups.
- **OffQ:** For quantization roadmaps, may be key to true 4-bit LLMs without mixed kernels.
- **Mirage-Persistent Kernel:** If someone has multi-GPU, MPK could drastically cut latency.

Weaker outliers: Sorted SpecDec (S2D) is niche to few-target scenarios; Clover-2’s gains are marginal for a small extra model.

Recommendations for Local Inference Stacks

- **Memory-Heavy Scenarios (long context):** Try HashEvict or CSKV if memory is the bottleneck. For extreme quant, use DecoQuant on KV, combined with standard 4-bit weight/activation quant.
- **Low-Latency/High-Throughput:** Investigate LazyLLM for heavy prompts to skip KV, and Token Recycling or CTC-Drafter to accelerate decoding.
- **Budget Hardware (CPU or moderate GPU):** Offloading less-frequent KV to CPU/SSD is still experimental; keep an eye on small embedded runtimes.
- **Multi-GPU:** If using more than one GPU, the Mirage MPK approach is worth

exploration.

- **Prototype Experiments:** It's worth integrating at least one "deep" trick into your framework, e.g. patching your attention loop for HashEvict eviction, or adding LazyLLM's token mask.
- **Benchmark Wisely:** Many obscure methods shine only on certain tasks. Test on your workload, and mix methods cautiously as interactions can be unpredictable.

Source Index and Bibliography

1. Gould & van den Hengel, "Back from the Future: Counter-Causal Surprise KV Eviction" (2026).
2. Liu, Huang et al., "HashEvict: Pre-Attention KV Eviction via LSH" (2024).
3. Wang et al., "CSKV: Channel Shrinking for KV Cache" (NeurIPS workshop 2024).
4. Fu et al., "LazyLLM: Dynamic Token Pruning" (arXiv 2024).
5. Wen et al., "Speculative Decoding with CTC-based Draft" (arXiv 2024).
6. Xiao et al., "Clover-2: RNN Speculative Decoding" (arXiv 2024).
7. Kavehzadeh et al., "S2D: Sorted Speculative Decoding" (arXiv 2024).
8. Christopher et al., "SpecDiff: Speculative Diffusion Decoding" (NAACL 2025).
9. Luo et al., "Token Recycling for LLM Inference" (ACL 2025).
10. Liu, Zhao et al., "DecoQuant: KV Cache Quantization via Decomposition" (arXiv 2024).
11. Zhang et al., "PQCache: Product Quantization for KV Cache" (arXiv 2024).
12. Chang et al., "Palu: Low-Rank Projection of KV Cache" (ICLR 2025 poster).
13. Zhang & Shen, "FDC: Fast KV Dimensionality Compression" (arXiv 2025).
14. Wu et al., "OffQ: Outlier Offset Quantization" (arXiv 2026).
15. Zhihao Jia et al., "Mirage: Persistent Kernel for LLMs" (blog, 2025).

LocalHouseLLM Research — Internal Technical Compendium

Document prepared by LocalHouseLLM Research. All findings, ratings, and citations herein reflect internal review of primary and secondary sources current as of the report date. For internal / partner distribution.